TYIT SEM-V PRACTICAL

Artificial Intelligence Application Development And JIRA

Module 1

Introduction to AI and Intelligent agents

1. Design a simple rational agent by defining PEAS for vacuum cleaner environment and autonomous taxi. classify environment fully /partially observable, deterministic/stochastic, episodic/sequential. Implement a simple table-driven agent in Python.

Design of a Simple Rational Agent

1. What is a Rational Agent?

A **Rational Agent** is an intelligent agent that selects the action that maximizes its performance measure based on the percepts received from the environment and its knowledge.

A rational agent:

- Perceives the environment using sensors.
- Takes actions using actuators.
- Chooses the best action to achieve its goal.
- Improves performance by making rational decisions.

---

2. PEAS Description

PEAS stands for:

- **P** – Performance Measure
- **E** – Environment
- **A** – Actuators
- **S** – Sensors

A. Vacuum Cleaner Agent

PEAS Table

Component	Description
Performance Measure	Clean all rooms, minimize energy consumption, minimize time, avoid unnecessary movements
Environment	Two rooms (A and B), dirt present or absent
Actuators	Move Left, Move Right, Suck, No Operation
Sensors	Detect current location, detect whether current room is dirty

---

Working

The vacuum cleaner:

1. Checks its current room.
2. If the room is dirty → Clean it.
3. Otherwise move to another room.
4. Stop when both rooms are clean.

B. Autonomous Taxi Agent

PEAS Table

Component	Description
Performance Measure	Reach destination safely, obey traffic rules, minimize travel time, minimize fuel consumption, maximize passenger comfort
---	---
Environment	Roads, vehicles, pedestrians, traffic signals, weather, passengers
Actuators	Steering, Accelerator, Brake, Horn, Indicator, Gear
Sensors	Cameras, GPS, LiDAR, Radar, Ultrasonic sensors, Speedometer

---

Working

The taxi:

1. Detects surrounding traffic.
2. Plans the safest route.

3. Obeys traffic rules.
4. Avoids obstacles.
5. Drops passenger safely.

---

3. Environment Classification

Property	Vacuum Cleaner	Autonomous Taxi
Observability	Fully Observable	Partially Observable
Determinism	Deterministic	Stochastic
Episodic / Sequential	Sequential	Sequential

Explanation

Fully Observable

The agent has complete information about the environment.

Example: Vacuum cleaner knows

- Current room
- Whether dirty
- Current location

Hence **Fully Observable**.

Partially Observable

The agent cannot know everything.

Example: Taxi cannot predict

- Sudden pedestrian movement
- Driver behavior
- Weather changes
- Hidden vehicles

Hence **Partially Observable**.

Deterministic

Current action completely determines the next state.

Example:

Vacuum performs **Suck**

↓

Room becomes clean.

No uncertainty.

Stochastic

Outcome contains uncertainty.

Example:

Taxi applies brake.

But stopping distance depends on

- Rain
- Road condition
- Tyre grip
- Speed

Hence stochastic.

Sequential

Current decision affects future decisions.

Example:

Vacuum moves to Room B

↓

Next action depends on Room B condition.

Taxi turns left

↓

Next route changes.

Therefore both environments are **Sequential**.

---

4. Simple Table-Driven Agent

A **Table-Driven Agent** stores all possible percept sequences and the corresponding actions in a lookup table.

Instead of reasoning, it simply searches the table and performs the stored action.

Algorithm

1. Receive percept.
2. Append percept to history.
3. Look up the percept sequence in the table.
4. Return the corresponding action.

Python Program

```
# Table-Driven Agent Example


# Table containing percepts and actions
table = {
    ('A', 'Dirty'): 'Suck',
    ('A', 'Clean'): 'Move Right',
    ('B', 'Dirty'): 'Suck',
    ('B', 'Clean'): 'Move Left'
}


def table_driven_agent(percept):
    """Returns action based on current percept."""
    return table.get(percept, "No Operation")


# Example percepts
percepts = [
```

```
    ('A', 'Dirty'),
    ('A', 'Clean'),
    ('B', 'Dirty'),
    ('B', 'Clean')
]

print("Vacuum Cleaner Table-Driven Agent\n")

for percept in percepts:
    action = table_driven_agent(percept)
    print("Percept:", percept, "-> Action:", action)
```

Sample Output

Vacuum Cleaner Table-Driven Agent

Percept: ('A', 'Dirty') -> Action: Suck

Percept: ('A', 'Clean') -> Action: Move Right

Percept: ('B', 'Dirty') -> Action: Suck

Percept: ('B', 'Clean') -> Action: Move Left

---

5. Advantages of Table-Driven Agent

- Very simple to implement.
- Fast action lookup.
- Useful for small environments.
- Easy to understand for beginners.

---

6. Disadvantages of Table-Driven Agent

- Requires a very large lookup table for complex environments.
- Memory usage grows rapidly as the number of percept sequences increases.

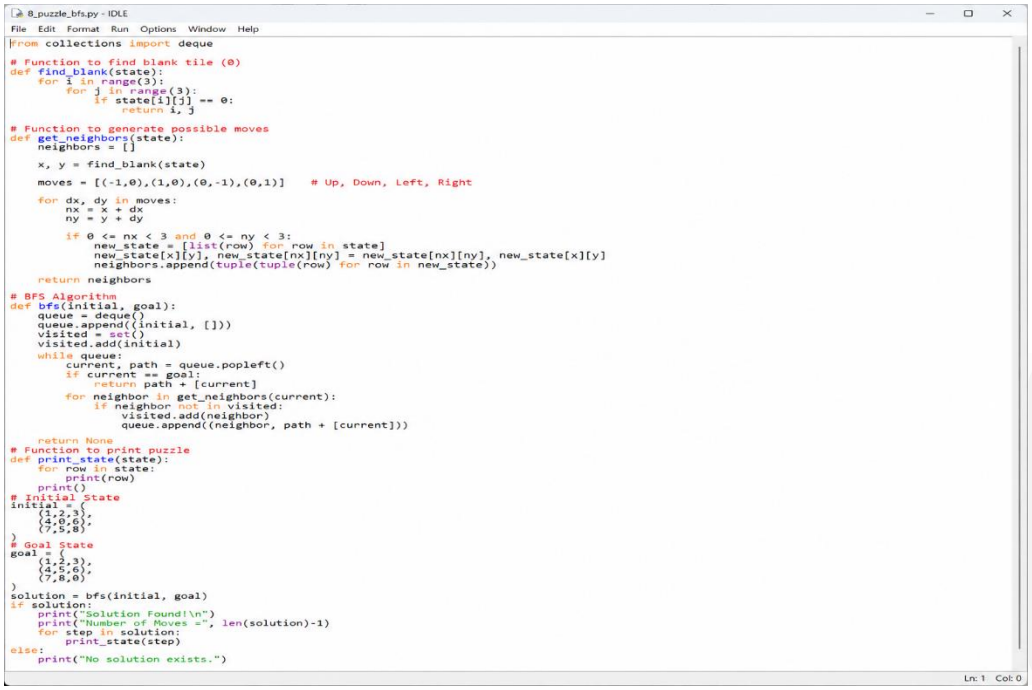
- Cannot learn from experience.
- Cannot handle new or unseen situations.
- Not suitable for real-world applications such as autonomous driving.

Problem Solving by Searching (Informed search)

1. Given an initial configuration of 8-puzzle and a goal configuration, Write a Python program to find the shortest sequence of moves to reach the goal state using Breadth-First search (BFS).

Algorithm

1. Represent the puzzle as a 3 × 3 matrix.
2. Store the initial state in a queue.
3. Maintain a set of visited states.
4. Remove the first state from the queue.
5. If it matches the goal state, print the solution path.
6. Otherwise, generate all valid neighbouring states by moving the blank tile (0).
7. Add unvisited neighbours to the queue.
8. Repeat until the goal state is found or the queue becomes empty.




```
Solution Found!

Number of Moves = 2

(1, 2, 3)
(4, 0, 6)
(7, 5, 8)

(1, 2, 3)
(4, 5, 6)
(7, 0, 8)

(1, 2, 3)
(4, 5, 6)
(7, 8, 0)
```

8-Puzzle using Breadth-First Search (BFS)

```
from collections import deque

# Function to find blank tile (0)
def find_blank(state):
    for i in range(3):
        for j in range(3):
            if state[i][j] == 0:
                return i, j

# Function to generate possible moves
def get_neighbors(state):
    neighbors = []

    x, y = find_blank(state)

    moves = [(-1,0), (1,0), (0,-1), (0,1)]   # Up, Down, Left, Right

    for dx, dy in moves:
        nx = x + dx
        ny = y + dy

        if 0 <= nx < 3 and 0 <= ny < 3:
            new_state = [list(row) for row in state]

            new_state[x][y], new_state[nx][ny] = new_state[nx][ny],
new_state[x][y]

            neighbors.append(tuple(tuple(row) for row in new_state))

    return neighbors

# BFS Algorithm
def bfs(initial, goal):

    queue = deque()
    queue.append((initial, []))

    visited = set()
    visited.add(initial)

    while queue:

        current, path = queue.popleft()

        if current == goal:
            return path + [current]

        for neighbor in get_neighbors(current):
```

```
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append((neighbor, path + [current]))

    return None

# Function to print puzzle
def print_state(state):
    for row in state:
        print(row)
    print()

# Initial State
initial = (
    (1,2,3),
    (4,0,6),
    (7,5,8)
)

# Goal State
goal = (
    (1,2,3),
    (4,5,6),
    (7,8,0)
)

solution = bfs(initial, goal)

if solution:
    print("Solution Found!\n")
    print("Number of Moves =", len(solution)-1)

    for step in solution:
        print_state(step)
else:
    print("No solution exists.")
```

2 .Given two water jugs of 4 lit and 3 lit capacity, write a python programm to obtain exactly 2 lits in one jug using depth first search, (vary capacity of jug.)

Algorithm

1. Represent each state as **(Jug1, Jug2)**.
2. Start from the initial state **(0, 0)**.
3. Generate all possible successor states:
    - Fill Jug1
    - Fill Jug2
    - Empty Jug1
    - Empty Jug2
    - Pour Jug1 → Jug2
    - Pour Jug2 → Jug1
4. Use a **stack** for DFS traversal.
5. Keep track of visited states.
6. Stop when either jug contains the target amount.
7. Print the sequence of states.

Python IDLE - Program Screenshot

Water_Jug_DFS.py - IDLE

File Edit Format Run Options Window Help

```
# Water Jug Problem using Depth First Search (DFS)

def get_next_states(state, jug1_cap, jug2_cap):
    x, y = state
    states = []

    # Fill Jug1
    states.append((jug1_cap, y))
    # Fill Jug2
    states.append((x, jug2_cap))
    # Empty Jug1
    states.append((0, y))
    # Empty Jug2
    states.append((x, 0))

    # Pour Jug1 -> Jug2
    transfer = min(x, jug2_cap - y)
    states.append((x - transfer, y + transfer))
    # Pour Jug2 -> Jug1
    transfer = min(y, jug1_cap - x)
    states.append((x + transfer, y - transfer))

  return states

def dfs(jug1_cap, jug2_cap, target):
    stack = [((0, 0), [])]
    visited = set()

    while stack:
        state, path = stack.pop()

        if state in visited:
            continue

        visited.add(state)
        path = path + [state]

        if target in state:
            return path
        for next_state in get_next_states(state, jug1_cap, jug2_cap):
            if next_state not in visited:
                stack.append((next_state, path))
    return None

# --------------- Main Program ----------------
jug1 = int(input("Enter capacity of Jug 1: "))
jug2 = int(input("Enter capacity of Jug 2: "))
target = int(input("Enter target amount: "))

solution = dfs(jug1, jug2, target)

if solution:
    print("\nSolution Found!")
    print("Sequence of States:\n")
    for state in solution:
        print(state)
else:
    print("\nNo Solution Exists.")
```

Ln: 57 Col: 29

Python IDLE - Output Screenshot

IDLE Shell 3.12.2

File Edit Shell Debug Options Window Help

```
Python 3.12.2 (tags/v3.12.2:6abddd9, Feb  6 2024, 21:26:36) [MSC v
.1937 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more informa
tion.
>>>
========== RESTART: C:/Users/HP/Desktop/Water_Jug_DFS.py =========
Enter capacity of Jug 1: 4
Enter capacity of Jug 2: 3
Enter target amount: 2

Solution Found!

Sequence of States:

(0, 0)
(4, 0)
(1, 3)
(1, 0)
(0, 1)
(4, 1)
(2, 3)

>>>
```

Water Jug Problem using Depth First Search (DFS)

Python Program

```
# Water Jug Problem using Depth First Search (DFS)

def get_next_states(state, jug1_cap, jug2_cap):
    x, y = state
    states = []

    # Fill Jug1
    states.append((jug1_cap, y))

    # Fill Jug2
    states.append((x, jug2_cap))

    # Empty Jug1
    states.append((0, y))

    # Empty Jug2
    states.append((x, 0))

    # Pour Jug1 -> Jug2
    transfer = min(x, jug2_cap - y)
    states.append((x - transfer, y + transfer))

    # Pour Jug2 -> Jug1
    transfer = min(y, jug1_cap - x)
    states.append((x + transfer, y - transfer))

    return states

def dfs(jug1_cap, jug2_cap, target):
    stack = [((0, 0), [])]
    visited = set()

    while stack:
        state, path = stack.pop()

        if state in visited:
            continue

        visited.add(state)
        path = path + [state]
```

```
        if target in state:
            return path

        for next_state in get_next_states(state, jug1_cap, jug2_cap):
            if next_state not in visited:
                stack.append((next_state, path))

    return None

jug1 = int(input("Enter capacity of Jug 1: "))
jug2 = int(input("Enter capacity of Jug 2: "))
target = int(input("Enter target amount: "))

solution = dfs(jug1, jug2, target)

if solution:
    print("\nSolution Found!\n")
    print("Sequence of States:\n")
    for state in solution:
        print(state)
else:
    print("\nNo Solution Exists.")
```

Sample Output

```
Enter capacity of Jug 1: 4
Enter capacity of Jug 2: 3
Enter target amount: 2
Solution Found!
Sequence of States:
(0, 0)
(4, 0)
(1, 3)
(1, 0)
(0, 1)
(4, 1)
(2, 3)
```

Problem Solving by Searching (Informed search)

1. Given an Initial configuration of the 8-puzzle and a goal configuration, write a python program to find the solution using Greedy Best-First search with Manhattan Distance heuristic.

Algorithm

1. Read the initial state.
2. Calculate the Manhattan Distance.
3. Insert the state into the priority queue.
4. Remove the state with the smallest heuristic value.
5. If it is the goal, stop.
6. Otherwise, generate all valid successor states.
7. Calculate the heuristic for each successor.
8. Insert successors into the priority queue.
9. Repeat until the goal is found.

```
from queue import PriorityQueue


# Goal State
goal = [[1,2,3],
    [4,5,6],
    [7,8,0]]


# Function to calculate Manhattan Distance
def heuristic(state):
  h = 0
  for i in range(3):
    for j in range(3):
      if state[i][j] != 0:
```

```
            x = (state[i][j]-1)//3
            y = (state[i][j]-1)%3
            h += abs(i-x) + abs(j-y)
    return h

# Function to find blank tile
def blank(state):
    for i in range(3):
        for j in range(3):
            if state[i][j] == 0:
                return i, j

# Greedy Best First Search
def greedy(start):
    pq = PriorityQueue()
    pq.put((heuristic(start), start))
    visited = []

    while not pq.empty():
        h, state = pq.get()

        print("\nCurrent State:")
        for row in state:
            print(row)
        print("Heuristic =", h)

        if state == goal:
            print("\nGoal Reached!")
```

```
            return

        visited.append(state)

        x, y = blank(state)

        # Up, Down, Left, Right
        moves = [(-1,0),(1,0),(0,-1),(0,1)]

        for dx, dy in moves:
            nx = x + dx
            ny = y + dy

            if 0 <= nx < 3 and 0 <= ny < 3:
                new = [row[:] for row in state]
                new[x][y], new[nx][ny] = new[nx][ny], new[x][y]

                if new not in visited:
                    pq.put((heuristic(new), new))

# Initial State
start = [[1,2,3],
         [4,0,6],
         [7,5,8]]

greedy(start)
```

Output

Current State:

[1, 2, 3]

[4, 0, 6]

[7, 5, 8]

Heuristic = 2

Current State:

[1, 2, 3]

[4, 5, 6]

[7, 0, 8]

Heuristic = 1

Current State:

[1, 2, 3]

[4, 5, 6]

[7, 8, 0]

Heuristic = 0

Goal Reached!

2. Given an initial configuration of the 8-puzzle and a goal configuration, Write a python program to find a shortest path using A* search with the Manhattan Distance Heuristic.

Algorithm

1. Read the initial state.
2. Calculate the Manhattan Distance.
3. Set path cost **g = 0**.
4. Compute **f = g + h**.
5. Insert the state into the priority queue.
6. Remove the state with the smallest **f** value.
7. If it is the goal, stop.
8. Otherwise, generate successor states.
9. Update **g**, calculate **h**, then compute **f = g + h**.
10. Repeat until the goal is reached.

```
from queue import PriorityQueue

# Goal State
goal = [[1,2,3],
     [4,5,6],
     [7,8,0]]

# Manhattan Distance Heuristic
def heuristic(state):
  h = 0
  for i in range(3):
    for j in range(3):
      if state[i][j] != 0:
        x = (state[i][j]-1)//3
        y = (state[i][j]-1)%3
        h += abs(i-x) + abs(j-y)
  return h

# Find Blank Tile
def blank(state):
  for i in range(3):
```

```
        for j in range(3):
            if state[i][j] == 0:
                return i, j

    # A* Search
    def astar(start):
        pq = PriorityQueue()

        # (f, g, state)
        pq.put((heuristic(start), 0, start))

        visited = []

        while not pq.empty():

            f, g, state = pq.get()

            print("\nCurrent State:")
            for row in state:
                print(row)
            print("g =", g, " h =", heuristic(state), " f =", f)

            if state == goal:
                print("\nGoal Reached!")
                return

            visited.append(state)

            x, y = blank(state)

            moves = [(-1,0),(1,0),(0,-1),(0,1)]

            for dx, dy in moves:

                nx = x + dx
                ny = y + dy

                if 0 <= nx < 3 and 0 <= ny < 3:

                    new = [row[:] for row in state]
                    new[x][y], new[nx][ny] = new[nx][ny], new[x][y]
```

```
            if new not in visited:

                new_g = g + 1
                new_f = new_g + heuristic(new)
                pq.put((new_f, new_g, new))

# Initial State
start = [[1,2,3],
      [4,0,6],
      [7,5,8]]

astar(start)
```

Output:

```
Current State:
[1, 2, 3]
[4, 0, 6]
[7, 5, 8]
g = 0  h = 2  f = 2

Current State:
[1, 2, 3]
[4, 5, 6]
[7, 0, 8]
g = 1  h = 1  f = 2

Current State:
[1, 2, 3]
[4, 5, 6]
[7, 8, 0]
g = 2  h = 0  f = 2

Goal Reached!
```

Bayes' Rule Application

A medical test is used to detect a particular disease. The probability that a randomly selected person has the disease is 1%. if a person has the disease, the test returns positive with probability 99%. If a person does not have the disease, the test still returns positive with probability 5%. (false positive rate). A person takes the test and receives s a positive result.

Problem statement: Write a python program to compute probability that the person actually has the disease given that the test result is positive.

Given

- Probability of disease: $P(D) = 0.01$
- Probability of no disease: $P(\neg D) = 0.99$
- Positive test if disease is present: $P(+|D) = 0.99$
- False positive rate: $P(+|\neg D) = 0.05$

We need to calculate:

$$P(D|+) = \frac{P(+|D) \times P(D)}{P(+|D) \times P(D) + P(+|\neg D) \times P(\neg D)}$$

```
# Python program to calculate the probability of having
# a disease given that the medical test result is positive

# Given probabilities
p_disease = 0.01
p_no_disease = 1 - p_disease

p_positive_given_disease = 0.99
p_positive_given_no_disease = 0.05

# Calculate total probability of getting a positive result
p_positive = (p_positive_given_disease * p_disease) + \
             (p_positive_given_no_disease * p_no_disease)

# Apply Bayes' Theorem
p_disease_given_positive = (p_positive_given_disease * p_disease) / p_positive

# Display the result
print("Probability of having disease given positive test:")
print(p_disease_given_positive)

# Convert probability into percentage
print("Percentage:", p_disease_given_positive * 100, "%")
```

Output

Probability of having disease given positive test:

0.16666666666666669

Percentage: 16.666666666666668 %

Naïve Bayes' Classification

Using a small dataset (e.g. weather or spam dataset), implement a Naive Bayes classifier using scikit learn.

```
# Import required libraries
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB

# Small spam dataset
messages = [
    "Win money now",
    "Free prize available",
    "Claim your free offer",
    "Meeting at 10 am",
    "Submit your assignment",
    "See you tomorrow"
]

# 1 = Spam, 0 = Not Spam
labels = [1, 1, 1, 0, 0, 0]

# Convert text into numbers
vectorizer = CountVectorizer()
X = vectorizer.fit_transform(messages)

# Create Naive Bayes model
model = MultinomialNB()
```



```
# Train the model
model.fit(X, labels)

# New message
new_message = ["Win a free prize"]

# Convert new message into numbers
new_data = vectorizer.transform(new_message)

# Predict
prediction = model.predict(new_data)

# Display result
if prediction[0] == 1:
    print("Spam Message")
else:
    print("Not Spam Message")
```

Output

```
Spam Message
```

Explanation of program

We first create a small dataset containing **spam and normal messages**.

labels = [1, 1, 1, 0, 0, 0]

Here:

1 = Spam

0 = Not Spam

CountVectorizer() converts the text messages into numerical data:

vectorizer = CountVectorizer()

X = vectorizer.fit_transform(messages)

We then create the Naive Bayes classifier:

model = MultinomialNB()

Train the classifier:

model.fit(X, labels)

Finally, we give a new message:

new_message = ["Win a free prize"]

The model predicts:

Spam Message

Conclusion

Thus, the **Naive Bayes classifier** was successfully implemented using scikit-learn to classify a given message as **Spam or Not Spam**.